

Table des matières

Cours 02 — Le langage SQL : lire et calculer	1
Objectifs	1
Théorie	2
Points clés à retenir.....	19
Exercices.....	20

Cours 02 — Le langage SQL : lire et calculer

Module 2/12 — Formation SQL avancé

Comment lire ce module — Le Cours 01 vous a appris à relier des tables et à vous méfier de NULL. Ce cours-ci ouvre la boîte à outils du langage lui-même : trier, filtrer, décider ligne par ligne, convertir, découper du texte, calculer sur des dates, compter juste. Chaque outil est montré sur trois petites tables neutres, une bibliothèque de quartier, qu'on lit d'un coup d'œil. Les deux laboratoires (Labo_02_OLAP.md et Labo_02_OLTP.md) reprennent tout sur AdventureWorks : c'est là que les pièges cessent d'être des exemples.

Où s'arrêter — Ce cours est, avec le Cours 03, l'un des deux plus gros de la formation : environ dix heures en tout, quatre séances de deux heures et demie. La théorie et les deux labos en prennent deux, les exercices et leur corrigé les deux autres. La coupure tombe après la section 3 : les sections 1 à 3 apprennent à sortir les bonnes lignes, les sections 4 à 7 à travailler les valeurs qu'elles portent. Les labos se coupent au même endroit, à la fin de leur partie 3.

Objectifs

À la fin de ce module, vous saurez :

- écrire une requête complète, avec ses expressions calculées, ses alias, son tri et sa limite, en sachant ce que le moteur garantit ou non sur l'ordre des lignes ;
- filtrer sans vous tromper de priorité entre AND et OR, et choisir entre BETWEEN, IN, LIKE et ILIKE ;
- décider d'une valeur ligne par ligne avec CASE, en plaçant les conditions dans le bon ordre ;
- convertir une valeur d'un type à un autre, et reconnaître les calculs que le type fausse en silence : la division entière, l'arrondi, les espaces de fin ;
- nettoyer, découper et recoller du texte ;
- extraire, tronquer, décaler et comparer des dates, et calculer un âge juste ;

- compter ce que vous croyez compter, et respecter les règles du GROUP BY.

Théorie

La bibliothèque de quartier

Tous les exemples de ce cours tournent sur trois tables, assez petites pour afficher chaque résultat en entier. Une bibliothèque prête des livres à ses membres. Les titres, les auteurs et les ISBN sont inventés.

La table membre :

membre_id	prenom	second_prenom	nom	courriel	date_naissance	ville
1	Sophie	NULL	Tremblay	sophie.tremblay@ courriel.test, précédé d'une espace	1988-03-14	Montréal
2	Émile	Jean	Gagnon	EMILE.GAGNON@COU RRIEL.TEST	2001-12-31	Québec
3	Léa	NULL	Roy	lea.roy@courriel .test	1975-07-02	Laval
4	Marc	André	Côté	marc.cote@courri el.test	1964-01-01	Montréal
5	Chloé	NULL	Bouchard	chloe.bouchard@c ourriel.test	2010-06-15	Sherbrooke
6	Minh	NULL	Nguyen	minh.nguyen@cour riel.test	1995-11-30	Montréal

La table livre :

livre_id	isbn	titre	auteur	genre	annee_parution	nb_pages	prix
1	978-2-9800001- 0-5	Le Fleuve en hiver	Lavoie, Hélène	Roman	1998	'312'	24,95
2	0-9690001-2-X	Petite histoire des ponts	Morin, Paul	Essai	1987	'180'	18,50

livre_id	isbn	titre	auteur	genre	annee_parution	nb_pages	prix
3	978-2-9800001-3-6	Chroniques du Plateau	Pelletier, Anne	roman	2005	'254'	21,00
4	978-2-9800001-4-3	Quatre saisons au chalet	Fortin, Jacques	BD	2012	'64'	29,95
5	978-2-9800001-5-0	Le Grand Livre des étoiles	Lavoie, Hélène	Jeunesse	2019	'inconnu'	34,99
6	978-2-9800001-6-7	Après la guerre	Morin, Paul	Roman	1975	'410'	16,95
7	978-2-9800001-7-4	Cuisiner sans recette	Nadeau, Julie	Pratique	2021	'208'	27,50
8	978-2-9800001-8-1	Guerre et paix au dépanneur	Pelletier, Anne	roman	2016	'198'	22,00

La table emprunt, où un livre se rend 21 jours après l'emprunt :

emprunt_id	membre_id	livre_id	emprunte_le	retour_prevu	rendu_le	amende
1	1	1	2026-01-05 10:15	2026-01-26	2026-01-20 14:00	0,00
2	2	4	2026-01-31 16:30	2026-02-21	2026-02-25 11:10	1,00
3	3	6	2026-02-02 09:05	2026-02-23	2026-02-23 17:45	0,00
4	1	3	2026-02-14 13:20	2026-03-07	2026-03-20 10:00	3,25
5	4	2	2026-03-01 11:00	2026-03-22	NULL	NULL
6	6	5	2026-03-15 15:40	2026-04-05	2026-04-02 12:30	0,00
7	5	5	2026-03-31 15:40	2026-04-21	2026-04-10 09:00	0,00
8	2	7	2026-04-06 10:00	2026-04-27	NULL	NULL
9	3	8	2026-04-11 14:15	2026-05-02	2026-05-09 16:00	1,75
10	6	1	2026-04-20 18:05	2026-05-11	2026-05-04 10:30	0,00

emprunt_id	membre_id	livre_id	emprunte_le	retour_prevu	rendu_le	amende
11	1	7	2026-05-02 10:40	2026-05-23	NULL	NULL
12	4	6	2026-05-16 13:00	2026-06-06	2026-05-19 09:15	0,00

Le type de chaque colonne compte autant que son contenu, et plusieurs pièges du cours viennent de là :

Colonnes	Type	Ce que le type change
emprunte_le, rendu_le	TIMESTAMP, une date et une heure	une heure non nulle déjoue les comparaisons à la journée (section 6)
retour_prevu, date_naissance	DATE, sans heure	deux dates se soustraient en jours (section 6)
isbn, nb_pages	VARCHAR, du texte	se trie comme du texte et ne se calcule pas sans conversion (sections 1 et 4)
prix, amende	NUMERIC(6,2) et NUMERIC(5,2), des nombres exacts	aucune erreur d'arrondi en virgule flottante (section 4)
membre_id, livre_id, annee_parution	INTEGER	une division entre entiers peut perdre sa partie décimale (section 4)

Pour que les résultats ne changent pas d'une lecture à l'autre, on fixe la date du jour au 20 mai 2026 : les exemples écrivent DATE '2026-05-20' là où une vraie requête écrirait CURRENT_DATE.

Ces trois tables ne sont pas qu'un dessin : le fichier monter_bibliotheque.sql, à côté de ce cours, les crée et les remplit, sur l'entrepôt comme sur le transactionnel. Le même script passe sur les deux moteurs, et il est rejouable autant de fois qu'on veut.

```
duckdb adventureworksdw.duckdb -f monter_bibliotheque.sql
PGCLIENTENCODING=UTF8 psql -X -d Adventureworks -f monter_bibliotheque.sql
```

Montez-les avant de lire la suite, et rejouez chaque exemple : tous les résultats affichés dans ce cours en sortent, et un cours dont on peut vérifier les tableaux vaut mieux qu'un cours qu'il faut croire.

1. L'anatomie d'une requête

Labo : Labo_02_OLTP.md et Labo_02_OLAP.md, partie 1.

Une requête de lecture a au plus sept clauses, SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY et LIMIT, et le Cours 01 a montré qu'elles ne se lisent pas dans l'ordre où on les écrit. Ce qui se passe à l'intérieur du SELECT mérite qu'on s'y arrête.

On y met des colonnes, mais aussi des expressions : un calcul, un appel de fonction, une constante. Le mot-clé AS donne un nom à la colonne produite, un **alias**.

```
SELECT titre, prix, prix * 1.05 AS prix_avec_tps
FROM livre
ORDER BY prix DESC
LIMIT 3;
```

titre	prix	prix_avec_tps
Le Grand Livre des étoiles	34,99	36,7395
Quatre saisons au chalet	29,95	31,4475
Cuisiner sans recette	27,50	28,8750

Trois choses se lisent dans ce résultat. Le calcul garde quatre décimales, parce qu'un nombre à deux décimales multiplié par un autre en donne quatre ; on arrondira en section 4. Le DESC trie du plus grand au plus petit, et ASC, l'ordre croissant, est le défaut quand on ne précise rien. Enfin LIMIT 3 coupe après le tri, et c'est ce qui fait de ces trois lignes un palmarès.

Piège — LIMIT sans ORDER BY rend trois lignes, pas les trois premières : une table n'a pas d'ordre (Cours 01, section 4). Et même avec un ORDER BY, les ex aequo se départagent au gré du moteur, d'une exécution à l'autre. Si le résultat doit être stable, ajoutez une clé de tri qui ne se répète jamais : ORDER BY prix DESC, livre_id.

DISTINCT retire les doublons du résultat, sur l'ensemble des colonnes du SELECT :

```
SELECT DISTINCT genre
FROM livre
ORDER BY genre;
```

Six genres sortent, dont roman et Roman. Pour le moteur, deux chaînes qui diffèrent par une majuscule sont deux valeurs différentes ; la section 2 dit comment les réunir.

DISTINCT n'est pas une clause de plus, et il a sa place dans l'ordre du Cours 01 : il s'applique sur le résultat du SELECT, à l'étape 5, donc après le WHERE et le GROUP BY, et avant le ORDER BY. Deux conséquences suivent de cette place. Il ne peut pas réduire le travail du moteur en amont, puisqu'il arrive quand tout est déjà calculé. Et il porte sur la ligne entière du SELECT, jamais sur une colonne : ajouter une colonne au SELECT peut faire réapparaître des lignes qu'il venait d'écarter.

Le tri place aussi les NULL quelque part, et pas toujours là où on les attend :

```
SELECT emprunt_id, rendu_le
FROM emprunt
ORDER BY rendu_le DESC;
```

Sur PostgreSQL, les trois emprunts pas encore rendus, 5, 8 et 11, arrivent en tête : il considère NULL comme plus grand que toute valeur, donc dernier en ordre croissant et premier en ordre décroissant. DuckDB les met en dernier dans les deux sens, et son réglage default_null_order le dit, à NULLS_LAST. La même

requête rend donc deux ordres différents, sans aucune erreur. Les clauses `NULLS FIRST` et `NULLS LAST` tranchent sur les deux moteurs : `ORDER BY rendu_le DESC NULLS LAST`.

Un nombre rangé dans une colonne de texte se trie, lui, comme du texte :

```
SELECT titre, nb_pages
FROM livre
ORDER BY nb_pages;
```

titre	nb_pages
Petite histoire des ponts	180
Guerre et paix au dépanneur	198
Cuisiner sans recette	208
Chroniques du Plateau	254
Le Fleuve en hiver	312
Après la guerre	410
Quatre saisons au chalet	64
Le Grand Livre des étoiles	inconnu

La BD de 64 pages arrive après le roman de 410. Rien n'est cassé : le texte se compare caractère par caractère, et « 4 » passe devant « 6 ». La conversion en nombre règle la question (section 4).

Dernier point, l'alias dans le `WHERE` :

```
SELECT titre, prix * 1.05 AS prix_ttc
FROM livre
WHERE prix_ttc > 25;
```

PostgreSQL refuse : « la colonne `prix_ttc` n'existe pas ». C'est l'ordre d'exécution du Cours 01, le `WHERE` passe avant le `SELECT`, et l'alias n'est pas encore né. DuckDB, lui, accepte la requête et rend quatre livres, parce qu'il résout les alias du `SELECT` avant de filtrer. C'est commode, mais ne comptez pas dessus dans du code appelé à voyager : répétez l'expression, `WHERE prix * 1.05 > 25`, ou nommez-la une fois pour toutes dans une CTE (Cours 05).

Portabilité — `LIMIT n` est une extension répandue, absente de la norme, qui écrit `FETCH FIRST n ROWS ONLY`. Nos deux terrains acceptent les deux écritures, mesuré, et Snowflake `LIMIT`, d'après sa documentation. Dans du code appelé à voyager, la forme normalisée est celle qui demande le moins de vérifications ; un moteur qui ne l'a pas a une écriture à lui, à lire dans sa documentation avant de porter la requête. Le rang des `NULL`, lui, n'est pas fixé par la norme du tout : elle

donne NULLS FIRST et NULLS LAST pour le dire, et l'écart qu'on vient de voir entre nos deux terrains montre ce que coûte le défaut. Écrivez la clause plutôt que d'aller lire, moteur par moteur, ce que chacun a choisi.

2. Filtrer juste

Labo : Labo_02_OLTP.md et Labo_02_OLAP.md, partie 2.

Le WHERE garde les lignes dont la condition est vraie. Les opérateurs se rangent en quelques familles :

Opérateur	Sens	Exemple
=, <>	égal, différent (!= est accepté presque partout)	genre <> 'BD'
<, <=, >, >=	l'ordre, sur des nombres, des dates ou du texte	prix >= 25
BETWEEN a AND b	entre a et b, bornes comprises	annee_parution BETWEEN 1990 AND 2010
IN (...)	égal à l'une des valeurs de la liste	genre IN ('Essai', 'Pratique')
LIKE, ILIKE	ressemble à un motif	titre LIKE 'Le %'
IS NULL, IS NOT NULL	absent, présent (Cours 01, section 8)	rendu_le IS NULL

Les conditions se combinent avec AND, OR et NOT, et ces trois mots ont une priorité, comme la multiplication sur l'addition : NOT d'abord, puis AND, puis OR. Voyez ce qu'elle fait d'une intention pourtant claire, les romans et les BD parus après 2000 :

```
SELECT titre, genre, annee_parution
FROM livre
WHERE genre = 'Roman' OR genre = 'BD' AND annee_parution > 2000;
```

titre	genre	annee_parution
Le Fleuve en hiver	Roman	1998
Quatre saisons au chalet	BD	2012
Après la guerre	Roman	1975

Deux romans du siècle dernier passent le filtre. Le moteur a lié le AND avant le OR, et il a compris « les romans de n'importe quelle année, ou bien les BD parues après 2000 ». Les parenthèses rétablissent l'intention :

```
SELECT titre, genre, annee_parution
FROM livre
WHERE (genre = 'Roman' OR genre = 'BD') AND annee_parution > 2000;
```

Il ne reste qu'une ligne, la BD de 2012. Les deux romans parus après 2000 manquent encore à l'appel, mais pour une autre raison : leur genre est écrit roman, en minuscules. On y vient plus bas.

À retenir — Dès qu'un OR côtoie un AND, mettez des parenthèses, même quand la priorité vous donne raison. La personne qui relira la requête ne devrait pas avoir à la connaître.

BETWEEN a ses bornes comprises, et c'est précisément ce qui piège sur un horodatage. Les emprunts de mars :

```
SELECT COUNT(*) FROM emprunt
WHERE emprunte_le BETWEEN '2026-03-01' AND '2026-03-31';      -- 2
```

```
SELECT COUNT(*) FROM emprunt
WHERE emprunte_le >= '2026-03-01' AND emprunte_le < '2026-04-01'; -- 3
```

La borne '2026-03-31' devient 2026-03-31 00:00 quand on la compare à une colonne horodatée. L'emprunt 7, fait le 31 mars à 15 h 40, se trouve après la borne et disparaît du premier compte. La seconde écriture, l'**intervalle demi-ouvert**, prend le premier jour inclus et le lendemain du dernier jour exclu : elle est juste sur une date comme sur un horodatage, et c'est celle qu'on garde par défaut.

IN teste l'appartenance à une liste, genre IN ('Essai', 'Pratique'), et rend deux livres. Son contraire NOT IN cache le piège vu au Cours 01 (section 8) : un seul NULL dans la liste, et la condition ne rend plus rien. Le Cours 07 y reviendra avec les sous-requêtes.

LIKE compare à un motif, où % remplace n'importe quelle suite de caractères, même vide, et _ exactement un caractère. isbn LIKE '_-%' trouve les ISBN dont le premier bloc n'a qu'un chiffre, ici le seul ISBN à dix chiffres de la table. Pour chercher un vrai % ou un vrai _, on déclare un caractère d'échappement : LIKE '%10!%%' ESCAPE '!' cherche le texte « 10% ».

La comparaison de texte est sensible à la casse sur les deux moteurs de la formation :

```
SELECT titre FROM livre WHERE titre LIKE '%guerre%';      -- Après La guerre
SELECT titre FROM livre WHERE titre ILIKE '%guerre%';      -- plus Guerre et paix au dépanneur
```

ILIKE ignore la casse. UPPER fait le même travail sur une égalité : genre = 'Roman' trouve deux livres, UPPER(genre) = 'ROMAN' en trouve quatre. Ni l'un ni l'autre n'ignore les accents, en revanche : prenom ILIKE 'emile' ne trouve personne, prenom ILIKE 'émile' trouve Émile.

Portabilité — ILIKE est une extension : nos deux terrains l'ont, mesuré, et Snowflake aussi d'après sa documentation. La forme qui voyage est UPPER(titre) LIKE UPPER('%guerre%'). Mais la question de fond est ailleurs : la sensibilité à la casse n'est pas une propriété de LIKE, c'est celle de la collation, la règle de comparaison du texte attachée à la colonne ou à la base. Là où la collation ignore la casse, LIKE l'ignore aussi, et genre = 'Roman' rend les quatre livres sans qu'on ait rien demandé. Avant de porter une comparaison de texte, lisez donc la collation du terrain d'arrivée plutôt que le nom de son moteur.

3. Décider ligne par ligne : CASE

Labo : Labo_02_OLTP.md et Labo_02_OLAP.md, sous-partie 3.1.

CASE produit une valeur différente selon la ligne. Sa forme la plus utile, le CASE **cherché**, aligne des conditions :

```

SELECT emprunt_id,
       CASE
         WHEN rendu_le IS NOT NULL THEN 'rendu'
         WHEN retour_prevu < DATE '2026-05-20' THEN 'en retard'
         ELSE 'en cours'
       END AS statut
FROM emprunt
ORDER BY emprunt_id;

```

Neuf emprunts sortent « rendu », deux « en retard » (les emprunts 5 et 8, dont l'échéance est passée) et un « en cours » (le 11, attendu le 23 mai).

Le moteur évalue les WHEN de haut en bas et s'arrête au premier qui est vrai. L'ordre fait donc partie de la logique. Inversez les deux premières lignes, en testant l'échéance avant le retour, et le résultat devient faux sans la moindre erreur : dix emprunts « en retard », un seul « rendu ». Un livre rendu dans les temps a lui aussi une échéance passée, et c'est la première condition vraie qui gagne.

Sans ELSE, une ligne qui ne remplit aucune condition reçoit NULL :

```

SELECT titre, prix,
       CASE WHEN prix > 30 THEN 'cher' END AS etiquette
FROM livre
ORDER BY prix DESC
LIMIT 3;

```

Seul le livre à 34,99 \$ porte l'étiquette ; les deux suivants ont une étiquette vide. C'est parfois voulu, par exemple dans l'agrégat conditionnel du Cours 06, et parfois un oubli.

La forme **simple** compare une seule expression à des valeurs, CASE genre WHEN 'BD' THEN ... END. Elle est plus courte, mais elle cache un piège, parce qu'elle teste l'égalité et qu'une égalité avec NULL n'est jamais vraie :

```

SELECT prenom,
       CASE second_prenom WHEN NULL THEN 'aucun' ELSE second_prenom END AS forme_simple,
       CASE WHEN second_prenom IS NULL THEN 'aucun' ELSE second_prenom END AS forme_cherchee
FROM membre
WHERE membre_id <= 4;

```

prenom	forme_simple	forme_cherchee
Sophie	NULL	aucun
Émile	Jean	Jean
Léa	NULL	aucun

prenom	forme_simple	forme_cherchee
Marc	André	André

La forme simple ne dit jamais « aucun ». Pour tester une absence, seule la forme cherchée, avec `IS NULL`, fonctionne.

Deux raccourcis valent d’être connus. `COALESCE` et `NULLIF` (Cours 01, section 8) sont des `CASE` abrégés : `COALESCE(a, b)` vaut `CASE WHEN a IS NOT NULL THEN a ELSE b END`. Et `GREATEST` et `LEAST` rendent la plus grande et la plus petite de plusieurs valeurs, avec une divergence qui mérite qu’on s’y arrête. Pour l’emprunt 5, pas encore rendu, `GREATEST(CAST(rendu_le AS DATE), retour_prevu)` vaut 2026-03-22 sur PostgreSQL comme sur DuckDB, qui ignorent le `NULL`.

Portabilité — Snowflake, d’après sa documentation, propage le `NULL` dans `GREATEST` et `LEAST` : la même expression y vaut `NULL`. Il offre aussi un `CASE` à deux branches, `IFF(condition, si_vrai, si_faux)`, dont DuckDB a l’équivalent sous le nom `IF`, mesuré, et que PostgreSQL n’a pas. Ce raccourci est une commodité que chaque moteur nomme à sa façon ; c’est le `CASE` complet qui est normalisé, et lui seul marche partout.

4. Types, conversions et nombres

Labo : Labo_02_OLTP.md et Labo_02_OLAP.md, sous-parties 3.2 et 3.3.

Chaque colonne a un type, et le type décide des opérations permises. Cinq familles suffisent :

Famille	Types courants	Pour quoi
texte	<code>VARCHAR(n)</code> , <code>TEXT</code> , <code>CHAR(n)</code>	noms, codes, identifiants qu’on ne calcule pas
nombre exact	<code>INTEGER</code> , <code>NUMERIC(p, s)</code> , aussi appelé <code>DECIMAL</code>	quantités, montants
nombre approché	<code>FLOAT</code> , <code>DOUBLE PRECISION</code>	mesures physiques, statistiques
date et heure	<code>DATE</code> , <code>TIMESTAMP</code>	la section 6
booléen	<code>BOOLEAN</code>	vrai, faux, ou <code>NULL</code>

Dans `NUMERIC(6, 2)`, 6 est le nombre total de chiffres et 2 le nombre de décimales. PostgreSQL et DuckDB ont des entiers de plusieurs tailles, dont `SMALLINT`, `INTEGER` et `BIGINT`, sur deux, quatre et huit octets. Snowflake, d’après sa documentation, simplifie : tous ses entiers sont des `NUMBER(38, 0)`.

`CAST(valeur AS type)` convertit. PostgreSQL, DuckDB et Snowflake acceptent aussi l’écriture abrégée `valeur::type`. La conversion échoue quand la valeur ne ressemble pas au type visé :

```
SELECT CAST(nb_pages AS INTEGER)
FROM livre;
```

PostgreSQL s'arrête au premier obstacle : « syntaxe en entrée invalide pour le type integer : inconnu ». DuckDB aussi : Could not convert string 'inconnu' to INT32. Toute la requête échoue pour une seule ligne. DuckDB propose une conversion tolérante, TRY_CAST(nb_pages AS INTEGER), qui rend NULL au lieu d'échouer ; Snowflake a la même. PostgreSQL n'a pas d'équivalent : on teste d'abord la forme de la valeur.

```
SELECT titre,  
       CASE WHEN nb_pages ~ '^[0-9]+$' THEN CAST(nb_pages AS INTEGER) END AS pages  
FROM livre  
ORDER BY pages NULLS LAST;
```

L'opérateur ~ teste une expression régulière, et '^[0-9]+\$' veut dire « rien que des chiffres, du début à la fin ». Converti, le livre de 64 pages reprend sa place en tête du tri, et le livre au nombre de pages inconnu passe en fin de liste.

Piège — TRY_CAST transforme une erreur bruyante en NULL silencieux. Un NULL produit par une conversion ratée ressemble trait pour trait à une donnée absente, et il disparaît des moyennes sans laisser de trace. Après une conversion tolérante, comptez les NULL qu'elle a fabriqués.

Tout ce qui ressemble à un nombre n'en est pas un. L'ISBN 0-9690001-2-X commence par un zéro et finit par une lettre : converti en entier, il perdrait l'un et ferait échouer la conversion sur l'autre. Un identifiant sur lequel on ne calcule jamais reste du texte, même s'il ne contient que des chiffres.

Le type décide aussi du résultat d'un calcul, et la **division entière** en est le premier piège :

```
SELECT 7 / 2           AS entiers,  
       7 / 2.0         AS avec_decimale,  
       CAST(7 AS NUMERIC) / 2 AS converti;
```

PostgreSQL rend 3, 3,5 et 3,5 : entre deux entiers, la division est entière et la partie décimale tombe. DuckDB rend 3,5 dans les trois cas : chez lui, / entre deux entiers donne un décimal, et c'est // qui fait la division entière. Le piège frappe là où on ne l'attend pas, dans un taux :

```
SELECT COUNT(rendu_le) / COUNT(*) AS taux,  
       ROUND(100.0 * COUNT(rendu_le) / COUNT(*), 1) AS pourcentage  
FROM emprunt;
```

Neuf livres rendus sur douze, et un taux de 0 sur PostgreSQL, contre 0,75 sur DuckDB. COUNT rend un entier, et 9 divisé par 12 en entiers donne 0. La seconde colonne donne 75,0 sur les deux moteurs : multiplier d'abord par 100.0, un nombre à décimale, fait passer tout le calcul en décimal, et c'est l'écriture qui voyage.

Les nombres exacts et les nombres approchés ne calculent pas pareil non plus. $0.1 + 0.2 = 0.3$ est vrai en NUMERIC et faux en FLOAT, où la somme vaut 0,30000000000000004 : le FLOAT range ses valeurs en binaire, et 0,1 ne s'y écrit pas exactement. Un montant se range donc en NUMERIC.

L'arrondi, enfin, dépend du type et du moteur :

Appel	PostgreSQL	DuckDB	Pourquoi, sur PostgreSQL
ROUND(2.5)	3	3	en NUMERIC, la moitié s'arrondit vers l'extérieur
ROUND(2.5::FLOAT)	2	3	en FLOAT, la moitié va au chiffre pair
ROUND(24.95 * 1.05, 2)	26,20	26,20	deux décimales demandées
ROUND(24.95 * 1.05::FLOAT, 2)	erreur	26,2	PostgreSQL n'arrondit un FLOAT qu'à l'unité

DuckDB arrondit la moitié vers l'extérieur dans les deux types, et il accepte un nombre de décimales sur un FLOAT. Sur PostgreSQL, on convertit d'abord en NUMERIC.

Quatre fonctions mathématiques reviennent dans la suite de la formation :

```
SELECT prix,
       ROUND(prix * POWER(1.05, 3), 2) AS apres_trois_hausses,
       ROUND(LN(26.20 / 24.95), 4)   AS rendement_log
FROM livre
WHERE livre_id = 1;
```

Le prix de 24,95 \$ passe à 28,88 \$ après trois hausses successives de 5 % : POWER(x, n) élève x à la puissance n. LN donne le logarithme naturel, et LN du rapport entre deux prix donne le rendement logarithmique, ici 0,0489 ; EXP fait le chemin inverse. Le Cours 09 en fait un outil de travail sur les séries de prix. ABS, enfin, rend la valeur absolue, pour un écart dont le signe ne compte pas.

Un dernier type réserve une surprise, le texte à longueur fixe. En CHAR(3), la valeur 'LB ' est complétée d'espaces, et PostgreSQL ignore ces espaces à la comparaison : 'LB '::CHAR(3) = 'LB' est vrai. En VARCHAR, chaque espace compte : 'LB '::VARCHAR(3) = 'LB' est faux. DuckDB accepte l'écriture CHAR(3) mais en fait un VARCHAR : 'LB '::CHAR(3) = 'LB' y est faux aussi. Le labo OLAP montre ce que ça coûte sur une vraie colonne.

Portabilité — La division de deux entiers n'a pas de réponse unique, et la norme laisse la précision du résultat à l'implémentation : un moteur peut garder le type des opérandes et rendre un quotient entier, comme PostgreSQL, ou promouvoir en décimal, comme DuckDB. Ne pariez sur ni l'un ni l'autre, faites passer le calcul en décimal vous-même, comme la requête du taux plus haut. TRY_CAST est une extension, que l'entrepôt a et que PostgreSQL n'a pas ; là où le moteur d'arrivée ne l'a pas non plus, la parade est un CASE qui écarte les valeurs inconvertis avant de convertir. Le raccourci ::, enfin, est une commodité de PostgreSQL que DuckDB a reprise : CAST est la forme normalisée, et la seule sûre dans du code qui voyage.

5. Travailler le texte

Labo : Labo_02_OLTP.md et Labo_02_OLAP.md, partie 4.

Les fonctions de texte les plus courantes sont les mêmes sur les deux moteurs :

Fonction	Ce qu'elle fait	Exemple	Résultat
LENGTH(s)	le nombre de caractères	LENGTH('Émile')	5
UPPER(s), LOWER(s)	majuscules, minuscules, accents compris	UPPER('Émile')	ÉMILE
TRIM(s)	retire les espaces aux deux bouts	TRIM(' Laval ')	Laval
SUBSTRING(s, début, longueur)	extraît un morceau ; on compte à partir de 1	SUBSTRING('Lavoie, Hélène', 1, 6)	Lavoie
POSITION(motif IN s)	la position de la première occurrence, 0 si absente	POSITION(',', IN 'Lavoie, Hélène')	7
REPLACE(s, ancien, nouveau)	remplace toutes les occurrences	REPLACE('0-9690001-2-X', '-', '')	096900012X
SPLIT_PART(s, séparateur, n)	le n-ième morceau	SPLIT_PART('Lavoie, Hélène', ',', 2)	Hélène, avec l'espace

Deux chaînes se collent avec l'opérateur ||, et c'est là que NULL fait le plus de dégâts. Le nom complet des membres :

```
SELECT prenom || ' ' || second_prenom || ' ' || nom AS avec_barres,
       prenom || COALESCE(' ' || second_prenom, '') || ' ' || nom AS portable
FROM membre;
```

avec_barres	portable
NULL	Sophie Tremblay
Émile Jean Gagnon	Émile Jean Gagnon
NULL	Léa Roy
Marc André Côté	Marc André Côté
NULL	Chloé Bouchard
NULL	Minh Nguyen

Quatre noms sur six disparaissent de la première colonne : collé à NULL, tout devient NULL. La seconde écriture règle le cas en deux temps. ' ' || second_prenom vaut NULL quand le second prénom manque, et COALESCE le remplace alors par une chaîne vide. Elle marche telle quelle sur les deux moteurs.

PostgreSQL et DuckDB offrent aussi `CONCAT_WS(' ', prenom, second_prenom, nom)`, qui saute les NULL et rend les six noms. Tous les moteurs ne le font pas : l'écriture par `COALESCE`, elle, voyage partout.

Nettoyer, c'est souvent enchaîner deux fonctions. Le courriel de Sophie commence par une espace, celui d'Émile est en majuscules, et `LOWER(TRIM(courriel))` les ramène tous les deux à une forme comparable : la longueur du premier passe de 30 à 29 caractères.

Découper, c'est choisir le bon repère. L'auteur est écrit « Nom, Prénom », et l'affichage veut « Prénom Nom » :

```
SELECT auteur,  
       TRIM(SPLIT_PART(auteur, ',', 2)) || ' ' || SPLIT_PART(auteur, ',', 1) AS auteur_affiche  
FROM livre;
```

« Lavoie, Hélène » devient « Hélène Lavoie ». Le `TRIM` n'est pas décoratif : le second morceau commence par l'espace qui suivait la virgule, et sans lui l'affichage commencerait par un blanc.

Trois remarques pour finir. Une chaîne vide '' n'est pas NULL : c'est une valeur connue, qui ne contient rien. `LENGTH('Émile')` vaut 5, mais le compte d'octets vaut 6, `OCTET_LENGTH` sur PostgreSQL et `STRLEN` sur DuckDB, parce qu'un caractère accentué occupe deux octets en UTF-8 ; PostgreSQL compte en caractères la longueur d'un `VARCHAR(n)`, et DuckDB ne la vérifie pas du tout (Cours 03). Et la barre oblique inverse, `\`, est un caractère ordinaire dans un littéral, sur PostgreSQL comme sur DuckDB : `'\'` est une chaîne d'un caractère, `'\\'` en compte deux. Le labo OLAP en fait l'expérience.

Portabilité — Snowflake, d'après sa documentation, se sert de la barre oblique inverse comme caractère d'échappement dans un littéral, et y écrit `'\\'` pour une barre seule. Le même code change donc de sens d'un moteur à l'autre, sans erreur. `SUBSTR` est un synonyme de `SUBSTRING` sur les deux terrains. Pour le reste, deux repères valent mieux qu'une liste de moteurs. `||` est l'opérateur de concaténation de la norme, mais un moteur qui s'en sert déjà comme « ou » logique ne peut pas l'avoir, et `CONCAT` est alors la forme qui voyage ; `SPLIT_PART` est une extension, que l'on remplace, là où elle manque, par `SUBSTRING` et `POSITION`. Une dernière question se pose avant tout portage de texte, et elle ne se devine pas : le moteur d'arrivée distingue-t-il la chaîne vide du NULL ? Nos deux terrains les distinguent, mesuré ; un moteur qui les confond change le sens de toutes les conditions `IS NULL` du code porté.

6. Dates et heures

Labo : Labo_02_OLTP.md et Labo_02_OLAP.md, partie 5.

Deux types portent l'essentiel du travail. `DATE` range un jour, `TIMESTAMP` un jour et une heure. Les fuseaux horaires ajoutent une couche : PostgreSQL distingue `TIMESTAMP`, sans fuseau, de `TIMESTAMPTZ`, qui en a un, comme DuckDB, et l'entrepôt de la formation range ses dates en `TIMESTAMP`, sans fuseau. Tant qu'une colonne ne dit rien de son fuseau, traitez-la comme une heure locale et ne la convertissez pas.

`CURRENT_DATE` donne la date du jour et `CURRENT_TIMESTAMP` l'instant présent, sur les deux moteurs. Nos exemples les remplacent par la date fixe du 20 mai 2026.

`EXTRACT` sort un morceau d'une date :

```
SELECT emprunt_id, emprunte_le,  
       EXTRACT(YEAR FROM emprunte_le) AS annee,
```

```

    EXTRACT(MONTH FROM emprunte_le) AS mois,
    EXTRACT(DOW FROM emprunte_le) AS jour_semaine
FROM emprunt
WHERE emprunt_id IN (1, 2, 7);

```

L'emprunt 1 tombe un lundi (1), le 2 un samedi (6), le 7 un mardi (2). Le numéro du jour de la semaine est une convention, et chacun a la sienne :

Qui compte	Dimanche	Lundi	Samedi
EXTRACT(DOW ...), PostgreSQL et DuckDB	0	1	6
EXTRACT(ISODOW ...), PostgreSQL et DuckDB, norme ISO	7	1	6
DAYOFWEEK(...), raccourci de DuckDB	0	1	6
ISODOW(...), raccourci de DuckDB	7	1	6
la colonne DayNumberOfWeek de DimDate	1	2	7

Préférez la version ISO, qui ne dépend d'aucun réglage, et vérifiez la convention d'une table de calendrier avant de vous en servir.

DATE_TRUNC ramène une date au début de sa période, et c'est l'outil du regroupement par mois :

```

SELECT DATE_TRUNC('month', emprunte_le) AS mois,
       COUNT(*) AS emprunts
FROM emprunt
GROUP BY DATE_TRUNC('month', emprunte_le)
ORDER BY mois;

```

Deux emprunts en janvier, deux en février, trois en mars, trois en avril, deux en mai. Sur un horodatage, le résultat reste un horodatage, 2026-01-01 00:00:00. Sur une colonne DATE, PostgreSQL réserve une surprise : DATE_TRUNC('month', retour_prevu) rend un horodatage avec fuseau, 2026-02-01 00:00:00-05, qu'il faut reconvertir, CAST(DATE_TRUNC('month', retour_prevu) AS DATE). DuckDB rend un horodatage sans fuseau, 2026-02-01 00:00:00, qu'on reconvertit de la même façon.

Ajouter une durée s'écrit de la même façon sur les deux terrains, et autrement sur Snowflake :

Besoin	PostgreSQL et DuckDB	Snowflake
21 jours plus tard	emprunte_le + INTERVAL '21 days'	DATEADD(day, 21, emprunte_le)
un mois plus tard	DATE '2026-01-31' + INTERVAL '1 month'	DATEADD(month, 1, '2026-01-31'::DATE)
un jour de plus sur une date	retour_prevu + 1, qui reste une date	DATEADD(day, 1, retour_prevu)

Ajouter un mois au 31 janvier ne donne pas le 31 février : les deux terrains s'arrêtent au dernier jour du mois, le 28 février en 2026. Ils rendent en plus un horodatage, 2026-02-28 00:00:00, dès qu'un INTERVAL entre dans le calcul.

Pour l'écart entre deux dates, PostgreSQL et DuckDB soustraient : deux dates donnent un nombre de jours, deux horodatages un intervalle. DuckDB offre aussi DATEDIFF('day', debut, fin), que Snowflake écrit sans guillemets autour de l'unité. Le retard de chaque livre rendu :

```
SELECT emprunt_id, retour_prevu, rendu_le,  
       CAST(rendu_le AS DATE) - retour_prevu AS jours_de_retard  
FROM emprunt  
WHERE rendu_le IS NOT NULL  
ORDER BY emprunt_id;
```

Les emprunts 2, 4 et 9 sont en retard de 4, 13 et 7 jours ; l'emprunt 3 a été rendu le jour même de l'échéance, avec un retard de 0, et les cinq autres, rendus en avance, sortent en négatif. Et c'est là que le mélange des types piège. Comparez un horodatage à une date sans conversion, rendu_le > retour_prevu, et vous trouvez quatre livres en retard au lieu de trois : le 23 février à 17 h 45 est bien postérieur au 23 février à minuit.

Le calcul d'un âge concentre tous ces pièges :

```
SELECT prenom, date_naissance,  
       EXTRACT(YEAR FROM AGE(DATE '2026-05-20', date_naissance)) AS age_reel,  
       EXTRACT(YEAR FROM DATE '2026-05-20') - EXTRACT(YEAR FROM date_naissance) AS age_naif  
FROM membre;
```

prenom	date_naissance	age_reel	age_naif
Sophie	1988-03-14	38	38
Émile	2001-12-31	24	25
Léa	1975-07-02	50	51
Marc	1964-01-01	62	62
Chloé	2010-06-15	15	16
Minh	1995-11-30	30	31

La soustraction des années donne un âge faux pour quatre membres sur six : tous ceux dont l'anniversaire tombe après le 20 mai. AGE compte les anniversaires réellement passés, et DuckDB l'a aussi, avec les mêmes âges. Son DATEDIFF('year', ...), en revanche, reproduit exactement l'erreur de la colonne age_naif, parce qu'il compte les passages au 1er janvier, pas les anniversaires. La correction ajoute un test : si l'anniversaire de l'année n'est pas encore arrivé, on retire un an. Le labo OLAP l'écrit en entier, parce que c'est la forme qu'on rencontre sur les moteurs qui n'ont que DATEDIFF.

Pour afficher ou relire une date, les deux moteurs ont des fonctions aux noms et aux codes différents. PostgreSQL écrit `TO_CHAR(emprunte_le, 'YYYY-MM-DD HH24:MI')`, qui donne `2026-01-31 16:30`, et `TO_DATE('20260131', 'YYYYMMDD')`, qui reconstruit la date à partir d'une clé comme celles d'un entrepôt. DuckDB écrit `STRFTIME(emprunte_le, '%Y-%m-%d %H:%M')` et `STRPTIME('20260131', '%Y%m%d')`, avec les codes à pourcentage du langage C, et `STRPTIME` rend un horodatage. Les noms de mois divergent aussi : PostgreSQL les écrit dans la langue du serveur avec le code `'TMMonth'`, DuckDB en anglais avec `'%B'`.

À retenir — Trois réflexes évitent l'essentiel des erreurs de dates. Filtrez une période par un intervalle demi-ouvert. Convertissez l'horodatage en date avant de le comparer à une date. Et ne calculez jamais un âge ou une ancienneté en soustrayant des années. Le Cours 09 fera de ces fonctions un outillage complet de séries temporelles.

7. Agréger avec rigueur

Labo : Labo_02_OLTP.md et Labo_02_OLAP.md, partie 6.

Le Cours 01 a posé `GROUP BY` et `HAVING`. Il reste à savoir exactement ce que chaque fonction compte :

```
SELECT COUNT(*)           AS emprunts,
       COUNT(rendu_le)     AS rendus,
       COUNT(DISTINCT membre_id) AS membres,
       COUNT(DISTINCT livre_id) AS livres
FROM emprunt;
```

Douze emprunts, neuf rendus, six membres, huit livres. `COUNT(*)` compte des lignes, `COUNT(colonne)` les valeurs présentes, et `COUNT(DISTINCT colonne)` les valeurs différentes. Trois questions, trois réponses.

`SUM`, `AVG`, `MIN` et `MAX` ignorent les `NULL`, et la moyenne en tire une conséquence :

```
SELECT SUM(amende)          AS total,
       AVG(amende)          AS moyenne,
       AVG(COALESCE(amende, 0)) AS moyenne_sur_tout
FROM emprunt;
```

Le total vaut 6,00 \$. La moyenne vaut 0,67 \$ sur les neuf emprunts clos, et 0,50 \$ si l'on compte les trois emprunts en cours comme sans amende. Ce sont deux questions différentes, et c'est à vous de choisir laquelle vous posez, pas au `NULL`.

La règle du `GROUP BY` tient en une phrase : toute colonne du `SELECT` qui n'est pas dans une fonction d'agrégat doit figurer dans le `GROUP BY`. Le moteur refuse sinon de choisir, pour chaque groupe, laquelle des valeurs afficher :

```
SELECT m.nom, m.ville, COUNT(*) AS emprunts
FROM emprunt AS e
JOIN membre AS m ON m.membre_id = e.membre_id
GROUP BY m.nom;
```

PostgreSQL refuse : « la colonne m.ville doit apparaître dans la clause GROUP BY ». Regrouper sur la clé primaire, GROUP BY m.membre_id, fait accepter la même requête à PostgreSQL : chaque membre n’a qu’une ville, et le moteur sait le déduire de la clé. DuckDB ne fait pas cette déduction et refuse encore, en suggérant ANY_VALUE(ville) ; il offre en revanche GROUP BY ALL, qui regroupe d’office sur toutes les colonnes non agrégées. Les deux moteurs acceptent aussi de regrouper par la position d’une colonne, GROUP BY 1, ou par son alias.

Le regroupement hérite enfin de tout ce que les sections précédentes ont dit. GROUP BY genre produit deux groupes de romans, roman et Roman, de deux livres chacun ; GROUP BY UPPER(genre) les réunit en un groupe de quatre. Et un compte n’est juste que si l’on sait ce qu’une ligne représente : dans une table de lignes de commande, COUNT(*) compte des lignes et COUNT(DISTINCT numero_de_commande) des commandes. Les deux labos mesurent l’écart sur AdventureWorks.

Rappel — Filtrer des lignes se fait dans le WHERE, filtrer des groupes dans le HAVING (Cours 01, section 6). Et calculer plusieurs mesures sur des sous-ensembles différents dans une même passe, c’est l’agrégat conditionnel, qu’enseigne le Cours 06.

8. Sur les deux terrains : OLTP et OLAP

Labo : Labo_02_OLTP.md, partie 7.

Le langage de ce cours est presque entièrement commun aux deux moteurs. Ce qui change, ce sont les données : leurs types, leurs conventions, et quelques fonctions qui n’ont pas la même logique.

Ce qu’on veut	Voie OLTP (PostgreSQL, Adventureworks)	Voie OLAP (DuckDB, adventureworksdw.duckdb)
Trier une colonne qui a des NULL	NULL en tête en ordre décroissant	NULL en dernier dans les deux sens
Filtrer un code d’une lettre	productline = 'R' marche : la colonne est un character, dont les espaces de fin ne comptent pas	ProductLine = 'R' ne trouve rien : la valeur chargée est 'R ', il faut un TRIM
Un taux entre deux comptes	division entière, 0 sans conversion	décimal d’office
Une conversion qui peut échouer	un CASE et une expression régulière	TRY_CAST
Le jour de la semaine	EXTRACT(ISODOW ...)	ISODOW, ou la colonne de DimDate, qui a sa propre convention
Un âge	AGE, juste d’emblée	AGE aussi ; DATEDIFF('year', ...) faux sans correction
Découper un identifiant domaine\nom	SPLIT_PART(loginid, '\', 2)	SPLIT_PART(LoginID, '\', 2), la même écriture

Ce qu'on veut	Voie OLTP (PostgreSQL, Adventureworks)	Voie OLAP (DuckDB, adventureworksdw.duckdb)
Une colonne non agrégée	acceptée si l'on regroupe par la clé primaire	refusée, sauf avec GROUP BY ALL

Deux leçons débordent le tableau.

La première, c'est que le type d'une colonne fait partie de la donnée. La même information, un code de gamme de produit, se compare juste d'un côté et échoue en silence de l'autre, parce qu'elle a été chargée dans un type différent. Avant de filtrer sur une colonne que vous ne connaissez pas, regardez sa longueur autant que ses valeurs distinctes.

La seconde, c'est que les deux bases ne portent pas les mêmes dates. L'entrepôt couvre les commandes de 2010 à 2014, le portage transactionnel les a décalées de 2022 à 2025. Les mêmes commandes, les mêmes montants, d'autres années : quand vous comparez un résultat d'un labo à l'autre, comparez des comptes et des montants, pas des dates.

Le même travail, deux fois — Le Cours 02 a deux laboratoires, Labo_02_OLAP.md et Labo_02_OLTP.md, qui posent les mêmes questions sur les deux terrains. Faites l'OLAP d'abord : le labo OLTP se termine sur le tableau des écarts, qui n'a de sens qu'une fois les deux faits.

Points clés à retenir

- Une expression du SELECT se nomme avec AS. LIMIT sans ORDER BY rend des lignes au hasard, et des ex aequo se départagent au hasard : une clé unique en fin de tri rend le résultat stable.
- Un nombre rangé en texte se trie comme du texte. NULL passe en dernier en ordre croissant sur PostgreSQL, en dernier dans les deux sens sur DuckDB : NULLS FIRST ou NULLS LAST lève le doute.
- NOT passe avant AND, qui passe avant OR : dès qu'un OR côtoie un AND, des parenthèses.
- Sur un horodatage, BETWEEN perd la dernière journée. L'intervalle demi-ouvert ($\geq$ début AND $<$ lendemain de la fin) est juste dans tous les cas.
- LIKE est sensible à la casse, ILIKE non, et aucun des deux n'ignore les accents.
- CASE évalue ses WHEN dans l'ordre et s'arrête au premier vrai ; sans ELSE, il rend NULL ; sa forme simple ne sait pas tester NULL.
- Une conversion ratée fait échouer la requête ; TRY_CAST la change en NULL silencieux, qu'il faut compter. Un identifiant qu'on ne calcule pas reste du texte.
- Entre entiers, PostgreSQL divise en entiers : $9 / 12$ vaut 0, et 0,75 sur DuckDB. Un montant se range en NUMERIC, et ROUND n'arrondit pas la moitié de la même façon selon le type et le moteur.
- Collé à NULL, un texte devient NULL : COALESCE(' ' || second_prenom, '') protège la concaténation sur les deux moteurs.

- Comparez des dates à des dates, filtrez par intervalle demi-ouvert, et calculez un âge avec AGE ou avec un DATEDIFF corrigé, jamais par soustraction d'années.
 - COUNT(*), COUNT(colonne) et COUNT(DISTINCT colonne) répondent à trois questions. Toute colonne non agrégée figure au GROUP BY, sauf avec GROUP BY ALL sur DuckDB.
-

Exercices

Les exercices de ce cours et leur corrigé vivent dans Exercices_corriges_02.md. Faites-les après les deux laboratoires : plusieurs énoncés supposent que vous ayez vu de vos yeux les pièges du ProductLine, de la division entière et du DATEDIFF.